

The C2 platform cannot talk directly to “a drone” in the abstract. It needs a physical communications path, a protocol endpoint, and an adapter that converts that protocol into the C2’s internal vehicle model.

For a MAVLink drone, the minimum chain is:

MERMAID

flowchart LR

```
FC["Flight controller<br/>PX4 / ArduPilot"]
AR["Air telemetry radio<br/>or IP modem"]
GR["Ground radio / network endpoint"]
GW["Edge gateway<br/>MAVLink adapter"]
C2["C2 platform"]
```

```
FC -->|"UART / Ethernet<br/>MAVLink"| AR
AR <-->|"RF / LTE / mesh / Wi-Fi"| GR
GR -->|"USB / Ethernet / UDP"| GW
GW <-->|"Normalized telemetry<br/>authorized commands"| C2
```

What hardware is actually required?

Minimal MAVLink configuration

On the drone:

- Flight controller such as a Pixhawk-class controller
- PX4 or ArduPilot
- GPS, IMU, compass and other flight sensors
- Air-side telemetry radio or IP modem
- Antenna
- Appropriate power supply
- Optional RC receiver for manual backup

On the ground:

- Matching ground-side radio
- USB, serial or Ethernet connection
- Laptop or edge computer
- MAVLink adapter or ground-control application
- C2 application

This can work without a companion computer. PX4 documents [TELEM1](#) as the usual telemetry-radio connection and [TELEM2](#) as the usual companion-computer connection. [PX4 architecture](#)

Production-style configuration

A more established autonomous/C2 product normally uses:

```
MERMAID
flowchart LR
    subgraph UAV["On the UAV"]
        FC["Safety-critical<br/>flight controller"]
        MC["Linux mission/<br/>companion computer"]
        DL["IP data-link radio,<br/>LTE or mesh modem"]
        P["EO/IR or other payload"]
        FC <--> |"MAVLink / DDS<br/>serial or Ethernet"| MC
        P <--> |"Ethernet / USB / CAN"| MC
        MC <--> DL
    end

    subgraph Ground["On the ground"]
        GD["Ground radio/modem"]
        EC["Rugged edge computer"]
        UI["Operator laptop/tablet"]
        GD <--> EC
        EC <--> UI
    end

    DL <--> |"Encrypted IP link"| GD
```

The flight controller retains responsibility for stable flight and safety. The companion computer handles:

- C2 connectivity
- MAVLink routing
- Payload control
- Video encoding
- Computer vision
- Autonomous mission logic
- Store-and-forward operation
- Communications failover
- Vendor adapter containers

PX4 explicitly recommends a companion computer when advanced processing is needed. It normally runs Linux and connects to the flight controller through serial or Ethernet using MAVLink or DDS. [PX4 companion-computer architecture](#)

Examples from established products

Auterion Skynode

Skynode combines much of the vehicle integration stack:

- PX4 flight controller
- Linux mission computer
- Ethernet, CAN and serial interfaces
- Camera/video processing
- LTE and radio connectivity
- MAVLink interface
- Ground-control software

Auterion Mission Control can discover a connected Skynode over USB networking, Wi-Fi or LTE. Their USB connection presents Skynode as a network device and uses DHCP before the ground application automatically discovers the vehicle. [Auterion connection documentation](#)

This is an integrated commercial version of:

TEXT

Pixhawk + companion computer + MAVLink router + modem + managed software

Picogrid

Picogrid illustrates the ground-side integration appliance model:

- Legion is its integration/data platform.
- Helios and related expeditionary nodes supply field power, communications and edge compute.
- Sensors and unmanned platforms connect locally to the node.
- The node translates and distributes their data to C2 systems.

Picogrid describes Helios as bringing compute and communications to the tactical edge, while Legion provides the common interface. [Picogrid architecture](#)

Anduril Lattice

Lattice deployments use distributed tactical edge computers hosting the integration software near sensors, vehicles and command posts. The hardware-specific system talks to a local Lattice integration, and Lattice distributes the normalized state over its data mesh. Its public SDK exposes gRPC and REST interfaces. [Lattice SDK](#)

The important pattern is that Lattice does not require every drone to speak the Lattice data model natively. An integration component translates the drone's native protocol.

Surtr ParallaxOS

Surtr does not publish a detailed hardware specification. Reporting describes its current deliverable as an edge-compute unit with ParallaxOS installed. Sensors connect to that unit through standards or vendor APIs.

Publicly confirmed Surtr interfaces are:

- SAPIENT
- Link 16
- Cursor-on-Target
- ATAK
- Custom vendor APIs

MAVLink remains plausible but publicly unconfirmed.

In practical terms, ParallaxOS probably requires:

- A computer capable of running its software at the site
- Ethernet or serial connectivity to sensors
- A receiver or modem for any wireless sensors
- Network access to the upstream C2
- Vendor adapters for each connected system
- Accurate sensor location, orientation and time configuration

It does not eliminate the physical sensor receiver, drone modem or ground radio.

The MAVLink handshake

Importantly, pairing the radios is not part of MAVLink. The radio or IP network must first establish connectivity.

Stage 1: physical and network connection

The two endpoints first establish a transport:

- A paired telemetry-radio link
- Ethernet
- Wi-Fi
- LTE
- Tactical mesh

- Serial cable during bench testing

The ground edge computer then receives MAVLink through something such as:

```
TEXT
COM5 at 57600 or 921600 baud
UDP 0.0.0.0:14550
TCP vehicle-address:5760
Ethernet interface
```

A MAVLink router can forward the same vehicle stream to Sentinel, a local pilot station and a recorder simultaneously.

Stage 2: vehicle discovery

The autopilot broadcasts `HEARTBEAT` messages.

A heartbeat supplies:

- System ID
- Component ID
- Vehicle type
- Autopilot family
- Flight mode
- Armed/disarmed flags
- Vehicle state

On ordinary RF telemetry links, heartbeats are commonly transmitted around once per second. Missing several heartbeats indicates that the connection has been lost. [MAVLink heartbeat specification](#)

Sentinel's MAVLink adapter would see:

```
TEXT
HEARTBEAT
sysid: 14
compid: 1
type: QUADROTOR
autopilot: PX4
mode: HOLD
state: STANDBY
```

It should then map `sysid=14` to a durable Sentinel vehicle identity. A MAVLink system ID alone is not a sufficient enterprise identity because it can be reconfigured or duplicated.

Stage 3: version and capability discovery

The adapter requests `AUTOPILOT_VERSION` and component metadata.

This establishes:

- MAVLink 1 or MAVLink 2
- Firmware version
- Hardware UID when exposed
- Supported mission protocol
- Supported positional-command format
- Parameter support
- File-transfer support
- Camera and gimbal components
- Vendor-specific dialects

MAVLink provides a component-metadata protocol so a ground application can discover capabilities without hard-coding every vehicle. [MAVLink component metadata](#)

Sentinel should compare the reported capability set with an allow-list:

TEXT

```
Expected vehicle: sentinel-drone-07
Expected autopilot: PX4
Required: MAVLink 2
Required: mission-int
Required: command-int
Optional: camera, gimbal, FTP
```

Stage 4: authentication and encryption

MAVLink 2 can sign messages, allowing the receiver to verify that they came from someone with the shared secret.

However, MAVLink signing does not encrypt the data. [MAVLink security FAQ](#)

A production arrangement should therefore use:

- MAVLink 2 signing for message-level authenticity
- WireGuard, IPsec, radio encryption or another encrypted transport
- Device certificates or provisioned keys at the edge gateway
- An allow-list linking hardware identity to Sentinel vehicle identity
- Rotation and revocation of keys

Sentinel should not mark a vehicle **COMMANDABLE** merely because it emits a heartbeat.

Stage 5: time synchronization

The adapter and vehicle exchange **TIMESYNC** messages to estimate clock offset and link delay.

This is important for:

- Track association
- Video georegistration
- Sensor fusion
- Latency measurement
- Audit trails
- Replay

Without synchronization, “the drone was here at 14:03:01” could mean different times to the vehicle, camera and C2.

Stage 6: initial state synchronization

The adapter waits for or requests:

- Position
- Attitude
- Velocity
- GPS quality
- Home position
- Battery state
- Estimator health
- Current mode
- Armed status
- Current mission identifier
- Camera and payload state
- Radio/link quality

Only after these checks should the adapter transition through something like:

TEXT

DISCOVERED
→ AUTHENTICATED
→ SYNCHRONIZING
→ TELEMETRY_READY
→ COMMANDABLE

A drone may be visible but intentionally not commandable because:

- Firmware is unsupported
- Vehicle ID is unrecognized
- Signing failed
- Position estimate is unhealthy
- Time offset is unacceptable
- Another ground station owns control
- The safety pilot has not released authority

Stage 7: command handshake

Sentinel should send mission-level intent to its adapter:

JSON

```
{  
  "commandId": "cmd-9471",  
  "vehicleId": "sentinel-drone-07",  
  "type": "HOLD_POSITION",  
  "expiresAt": "2026-07-28T10:15:05Z"  
}
```

The adapter translates that into the appropriate MAVLink command addressed to the vehicle's system and component IDs.

MAVLink responds with **COMMAND_ACK**:

TEXT

ACCEPTED
IN_PROGRESS
DENIED
UNSUPPORTED
TEMPORARILY_REJECTED
FAILED

If an acknowledgement is not received, the command protocol requires retry behavior. "Accepted" only means that the flight stack accepted the command, not necessarily that the physical action has completed. [MAVLink command protocol](#)

For complete waypoint plans, the mission protocol performs a multi-message upload and finishes with [MISSION_ACK](#). [MAVLink mission protocol](#)

The sensor handshake is similar

A radar or acoustic sensor may not use MAVLink. It typically connects to an edge computer through:

- Ethernet
- Serial
- CAN
- USB
- ONVIF/RTSP
- A vendor SDK
- SAPIENT
- ASTERIX
- Cursor-on-Target

The edge adapter then performs:

```
MERMAID
sequenceDiagram
    participant Sensor
    participant Adapter
    participant Edge
    participant C2

    Sensor->>Adapter: Physical/IP connection
    Adapter->>Sensor: Query identity and capabilities
    Sensor-->>Adapter: Model, modalities, frames, task support
    Adapter->>Edge: Register source and sensor
    Edge-->>Adapter: Registration accepted
    Adapter->>Edge: Status and heartbeat
    Adapter->>Edge: Detections or local tracks
    Edge->>C2: Normalized observations
    C2->>Edge: Optional sensor task
    Edge->>Adapter: Look-at / mode / rate command
    Adapter->>Sensor: Vendor-native command
    Sensor-->>Adapter: Result
    Adapter-->>Edge: Command acknowledgement
```

SAPIENT formalizes this with registration, status reports, detection definitions, task definitions, detection reports and task acknowledgements.

Recommended Sentinel hardware architecture

For the first physical-drone bench integration:

On the drone

- PX4 or ArduPilot flight controller
- MAVLink-capable telemetry link
- Optional Linux companion computer
- Existing manual-control/failsafe path
- Camera only if needed for the demonstration

At the edge

- Laptop or small industrial PC
- Ground telemetry-radio endpoint
- Ethernet/Wi-Fi/LTE as appropriate
- `sentinel-mavlink-adapter`
- MAVLink Router
- Recorder and health monitor
- Device identity/key store

In Sentinel

```
TEXT
MAVLink adapter
↓
Sentinel Edge Gateway
↓
Canonical VehicleState
↓
C2 world model
↓
Policy and operator authority
↓
VehicleCommand
↓
Edge Gateway
↓
MAVLink adapter
↓
COMMAND_ACK / MISSION_ACK
```

The new adapter should be a separate process, just like the sensor simulator. Its southbound side speaks serial/UDP MAVLink, while its northbound side speaks Sentinel's authenticated edge contract.

The most important design rule is: **the UI must never own the MAVLink connection**. The edge adapter owns discovery, identity, synchronization, retries, command acknowledgements and failsafe behavior.

The UI only sees a durable Sentinel vehicle and requests authorized mission actions.